

Algorithms for Proposed Cryptosystem

1) Phase 1: Key Generation and Encapsulation

a) Algorithm 1: Receiver Knapsack Key Generation

Algorithm 1 yields the receiver's public and private key vectors using random integers, computing their sum, and securing them with a coprime multiplier. It replaces legacy RSA math with subset-sum (knapsack) approach, immediately resisting the initial handshake against Shor's Algorithm.

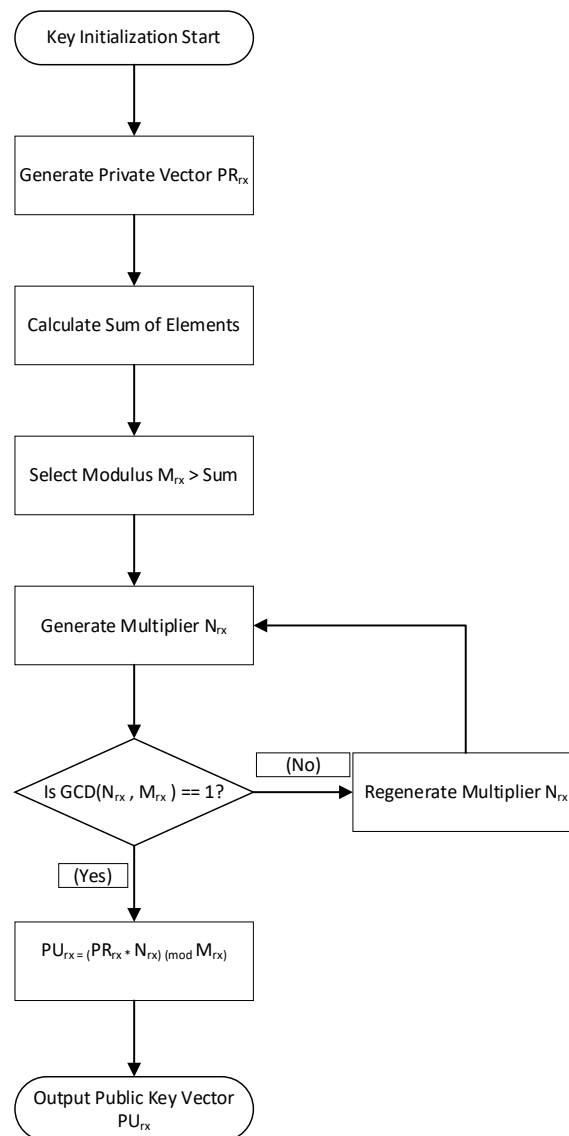


Figure 1 Flowchart of Receiver Key Initialization

ALGORITHM 1: RECEIVER KNAPSACK KEY GENERATION**Input:** Random Number Generator (RNG)**Output:** Public key Vector (PU_{rx}), Private Key Vector (PR_{rx}), Modulus (M_{rx}), Multiplier (N_{rx})

- 1: Initialize $PR_{rx} \leftarrow$ Array of 3 random integers $\in [5, 20]$
- 2: Compute $TotalSum \leftarrow \sum PR_{rx}$
- 3: Generate Modulus $M_{rx} \leftarrow TotalSum + random(5, 25)$
- 4: REPEAT
- 5: Generate Multiplier $N_{rx} \leftarrow$ random integer $\in [2, M_{rx} - 1]$
- 6: UNTIL $GreatestCommonDivisor(N_{rx}, M_{rx}) == 1$
- 7: // Public Key Transformation
- 8: FOR $i = 0$ to $length(PR_{rx}) - 1$ DO
- 9: $PU_{rx}[i] \leftarrow (PR_{rx}[i] * N_{rx}) \bmod (M_{rx})$
- 10: END FOR
- 11: RETURN $PU_{rx}, PR_{rx}, M_{rx}, N_{rx}$

b) Algorithm 2: Sender Knapsack Key Generation

Algorithm 2 describes the receiver's process to generate an independent set of public and private keys for the sender. It establishes the mathematical foundation required for the vector-based post-quantum digital signature used later in the process.

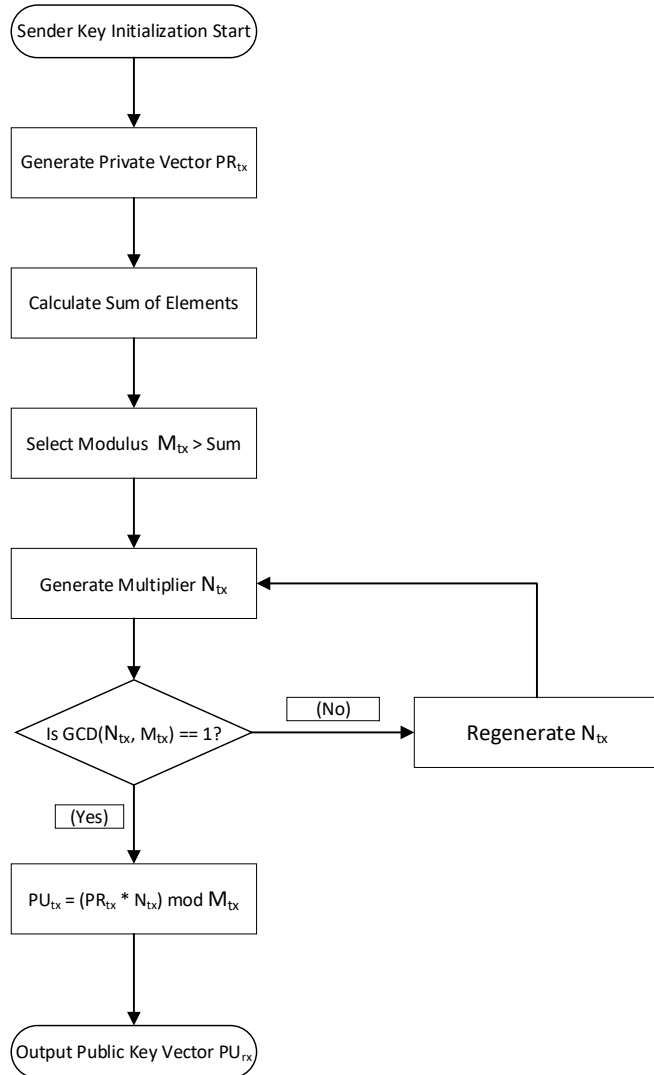


Figure 2 Flowchart of Sender Key Initialization

ALGORITHM 2: SENDER KNAPSACK KEY GENERATION**Input:** Random Number Generator (RNG)**Output:** Public key Vector (PU_{tx}), Private Key Vector (PR_{tx}), Modulus (M_{tx}), Multiplier (N_{tx})

- 1: Initialize $PR_{tx} \leftarrow$ Array of random integers
- 2: Compute $TotalSum \leftarrow \sum PR_{tx}$
- 3: Generate Modulus $M_{tx} \leftarrow TotalSum + \text{random}(5, 25)$
- 4: REPEAT
- 5: Generate Multiplier $N_{tx} \leftarrow \text{random integer} \in [2, M_{tx} - 1]$
- 6: UNTIL $\text{GreatestCommonDivisor}(N_{tx}, M_{tx}) == 1$
- 7: // Public Key Transformation
- 8: FOR $i = 0$ to $\text{length}(PR_{tx}) - 1$ DO
- 9: $PU_{tx}[i] \leftarrow (PR_{tx}[i] * N_{tx}) \bmod (M_{tx})$
- 10: END FOR
- 11: RETURN $PU_{tx}, PR_{tx}, M_{tx}, N_{tx}$

c) Algorithm 3: Chaotic Parameter Initialization

Algorithm 3 generates a binary session vector, two Caesar shift integers, and three floating-point coordinates to seed the Lorenz system. It creates the dynamic, ephemeral entropy required to ensure Perfect Forward Secrecy (PFS) and bypass the vulnerabilities of static keys.

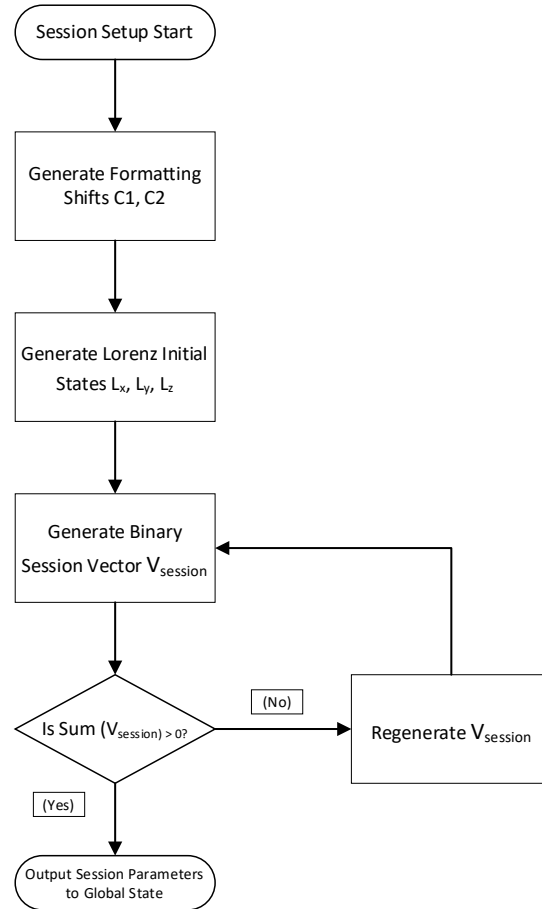


Figure 3 Flowchart of Ephemeral Session Initialization

ALGORITHM 3: CHAOTIC PARAMETER INITIALIZATION

Input: Random Number Generator (RNG)

Output: Session Vector (V_{session}), Shifts (C_1, C_2), Lorenz Initial States (L_x, L_y, L_z)

- 1: Generate Inner Shift $C_1 \leftarrow$ random integers $\in [1, 21]$
- 2: Generate Inner Shift $C_2 \leftarrow$ random integers $\in [1, 21]$
- 3: Generate $L_x \leftarrow$ random float $\in [0.0, 10.0)$
- 4: Generate $L_y \leftarrow$ random float $\in [0.0, 10.0)$
- 5: Generate $L_z \leftarrow$ random float $\in [0.0, 10.0)$
- 6: REPEAT
- 7: Initialize $V_{\text{session}} \leftarrow$ Array of 3 elements
- 8: FOR $i = 0$ to 2 DO
- 9: $V_{\text{session}}[i] \leftarrow$ random binary bit (0 or 1)
- 10: END FOR
- 11: VectorSum $\leftarrow \sum(V_{\text{session}})$
- 12: UNTIL VectorSum > 0
- 13: RETURN $V_{\text{session}}, C_1, C_2, L_x, L_y, L_z$

d) Algorithm 4: Session Vector Encapsulation

Algorithm 4 calculates a mathematical dot product between the binary session vector and the receiver's public key to create an encapsulated integer header. It secures the chaotic parameters for transmission across a network without triggering the computational bottlenecks of heavy modular exponentiation.

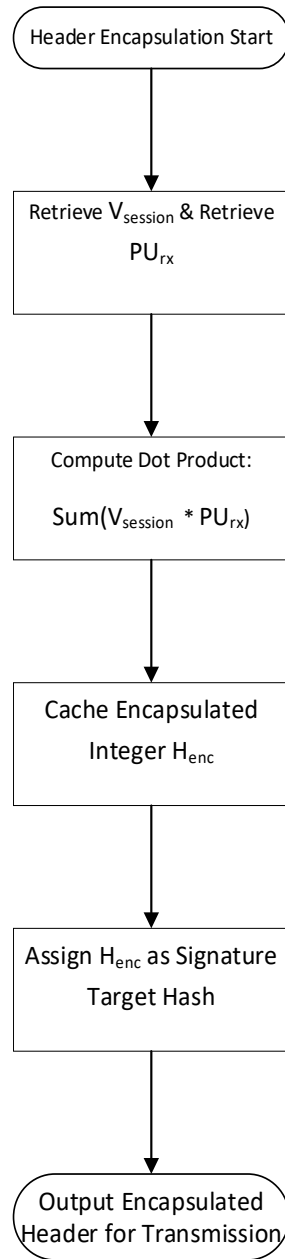


Figure 4 Flowchart of Key Encapsulation

ALGORITHM 4: SESSION VECTOR ENCAPSULATION**Input:** Session Vector (V_{session}), Receiver Public Key (PU_{rx})**Output:** Encapsulated Header Integer (H_{enc})

- 1: Initialize $H_{\text{enc}} \leftarrow 0$
- 2: // Vector Dot Product Calculation
- 3: FOR $i = 0$ to $\text{length}(V_{\text{session}}) - 1$ DO
- 4: $H_{\text{enc}} \leftarrow H_{\text{enc}} + (V_{\text{session}}[i] * PU_{\text{rx}}[i])$
- 5: END FOR
- 6: // Save for Phase 2 Digital Signature Generation
- 7: $\text{SignatureTargerHash} \leftarrow H_{\text{enc}}$
- 8: RETURN H_{enc}

2) Phase 2: Core Encryption

a) Algorithm 5: Plaintext Cleaning and Format Extraction

Algorithm 5 converts numbers to words forces uppercase formatting, and maps all standard punctuation into an extracted list. It cleanses the unpredictable natural language so it seamlessly fits into the strict mathematical boundaries of modulo-21 finite field.

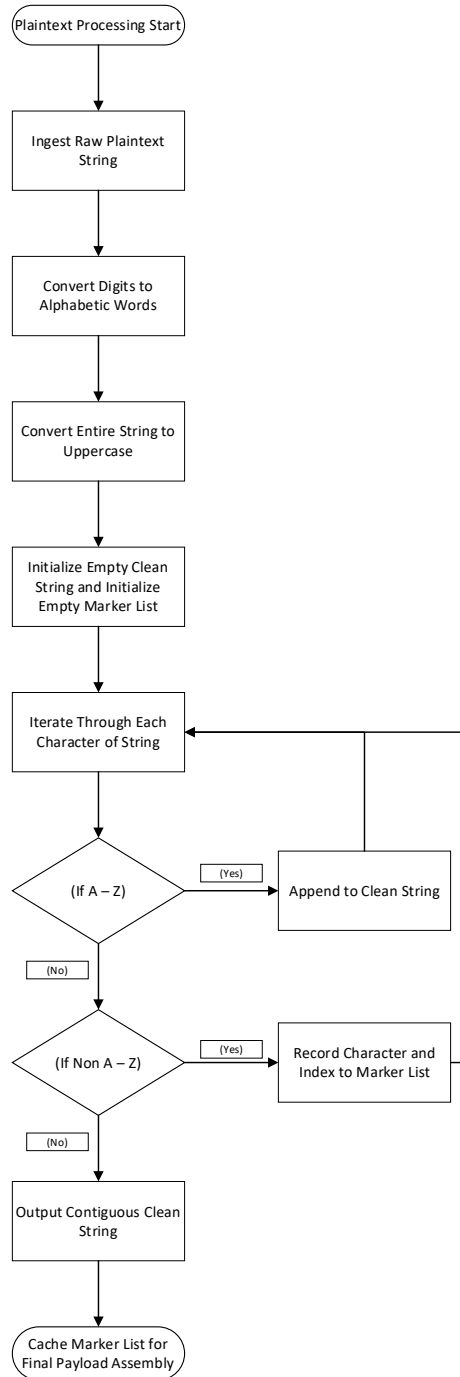


Figure 5 Flowchart of Plaintext Preparation

ALGORITHM 5: PLAINTEXT CLEANING AND FORMAT EXTRACTION**Input:** Raw Plaintext String (RawText)**Output:** Cleaned Uppercase String (CleanText), List of Formatting Markers (MarkerMap)

```
1: Raw Text ← ConvertNumbersToWords(RawText)
2: Raw Text ← ToUpperCase(RawText)
3: Initialize CleanText ← ""
4: Initialize MarkerMap ← Empty List
5: FOR i = 0 to length(RawText) – 1 DO
6:     CurrentChar ← RawText[i]
7:     IF CurrentChar ∈ ['A' ... 'Z'] THEN
8:         CleanText ← CleanText + CurrentChar
9:     ELSE
10:        NewMarker ← Create Object(Index: i, Character: CurrentChar)
11:        MarkerMap.Add(NewMarker)
12:    END IF
13: END FOR
14: RETURN CleanText, MarkerMap
```

b) Algorithm 6: First Shift and Alphabet Substitution

Algorithm 6 applies a modulo-26 Caesar shift and uses a dictionary mapping to convert the text into the specialized Enochian values. It flattens the statistical frequencies of English digraphs to defeat classical frequency analysis.

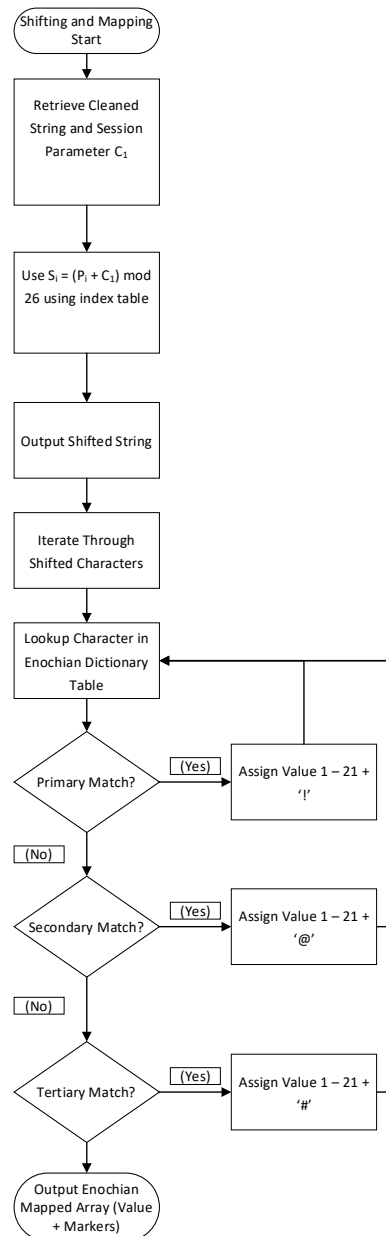


Figure 6 Flowchart of Shift and Homophonic Mapping

ALGORITHM 6: FIRST SHIFT AND ALPHABET SUBSTITUTION

Input: Cleaned String (CleanText), Shift Parameter (C_1), Dictionary (EnochianMap)

Output: Mapped Result Array (MappedArray)

```
1: Initialize ShiftedText  $\leftarrow$  ""
2: Initialize MappedArray  $\leftarrow$  Empty Array
3: // Stage 1: First Shift (Modulo 26)
4: FOR i = 0 to length(CleanText) - 1 DO
5:     CharIndex  $\leftarrow$  CleanText[i] - 'A'
6:     ShiftedIndex  $\leftarrow$  (CharIndex +  $C_1$ ) mod 26
7:     ShiftedText  $\leftarrow$  ShiftedText + (ShiftedIndex + 'A')
8: END FOR
9: // Stage 2: Homophonic Substitution
10: FOR i = 0 to length(ShiftedText) - 1 DO
11:     CurrentChar  $\leftarrow$  ShiftedText[i]
12:     // Lookup returns value (1-21) and Marker (!, @, #)
13:     DictionaryResult  $\leftarrow$  Lookup(EnochianMap, CurrentChar)
14:     MappedArray.Append(DictionaryResult.Value + DictionaryResult.Marker)
15: END FOR
16: RETURN MappedArray
```

c) Algorithm 7: Second Shift and Matrix Allocation

Algorithm 7 shifts the data using modulo-21 math and dynamically partitions the values into square matrices, padding empty spaces with zeros. It structures the data payload specifically to align with modern CPU L1/L2 caches for vectorized hardware execution.

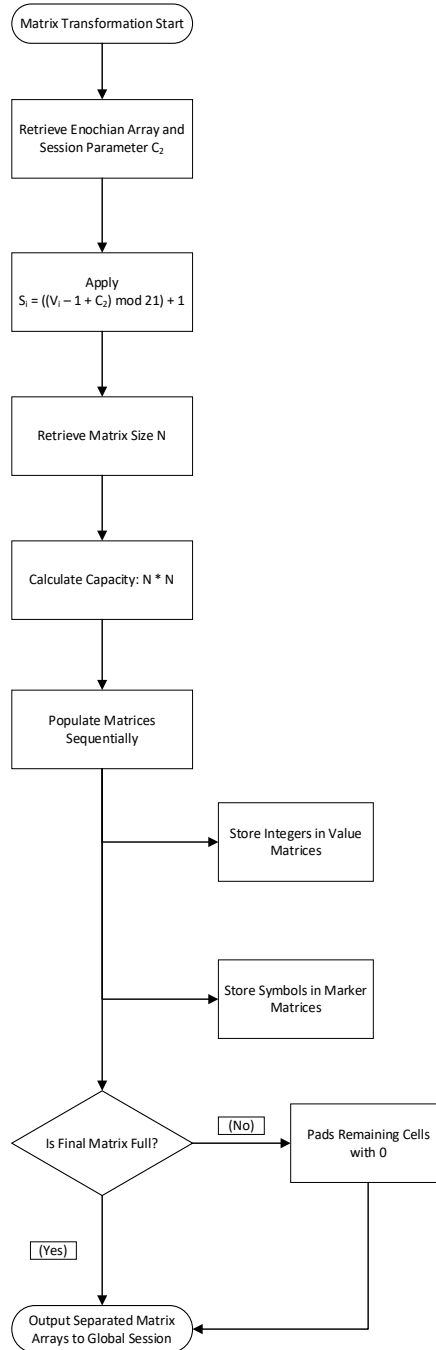


Figure 7 Flowchart of Matrix Construction

ALGORITHM 7: SECOND SHIFT AND MATRIX ALLOCATION**Input:** Mapped Array (MappedArray), Shift Parameter (C_2), Matrix Size (N)**Output:** List of Value Matrices (ValueList), List of Marker Matrices (MarkerList)

```
1:  // Stage 1: Second Shift (Modulo 21)
2:  FOR i = 0 to length(MappedArray) – 1 DO
3:      Split MappedArray[i] into Integer (V) and Marker (M)
4:       $New_V \leftarrow ((V - 1 + C_2) \bmod 21) + 1$ 
5:       $MappedArray[i] \leftarrow New_V + M$ 
6:  END FOR
7:  // Stage 2: Matrix Allocation
8:  Initialize ValueList  $\leftarrow$  Empty List
9:  Initialize MarkerList  $\leftarrow$  Empty List
10: Capacity  $\leftarrow N * N$ 
11: Index  $\leftarrow 0$ 
12: WHILE Index < length(MappedArray) DO
13:     Create Temp_Value_Matrix [N, N]
14:     Create Temp_Marker_Matrix [N, N]
15:     FOR row = 0 to N – 1 DO
16:         FOR col = 0 to N – 1 DO
17:             IF index < length(MappedArray) THEN
18:                 Extract Int (V) and Marker (M) from MappedArray[Index]
19:                 Temp_Value_Matrix [row, col]  $\leftarrow V$ 
20:                 Temp_Marker_Matrix[row, col]  $\leftarrow M$ 
21:                 Index  $\leftarrow$  Index + 1
22:             ELSE
23:                 Temp_Value_Matrix[row, col]  $\leftarrow 0$  // Zero Padding
24:                 Temp_Marker_Matrix[row, col]  $\leftarrow ""$ 
25:             END IF
```

```
26:         END FOR
27:     END FOR
28:     ValueList.Append(Temp_Value_Matrix)
29:     MarkerList.Append(Temp_Marker_Matrix)
30: END WHILE
31: RETURN ValueList, MarkerList
```


d) Algorithm 8: Chaotic Key Factor and Matrix Generation

Algorithm 8 uses Euler integration on the Lorenz ODEs to scale a base matrix, actively validating that its determinant is not a multiple of 3, 7, or 21. It ensures the encryption matrix is mathematically invertible while continuously mutating it to defeat KPA attacks.

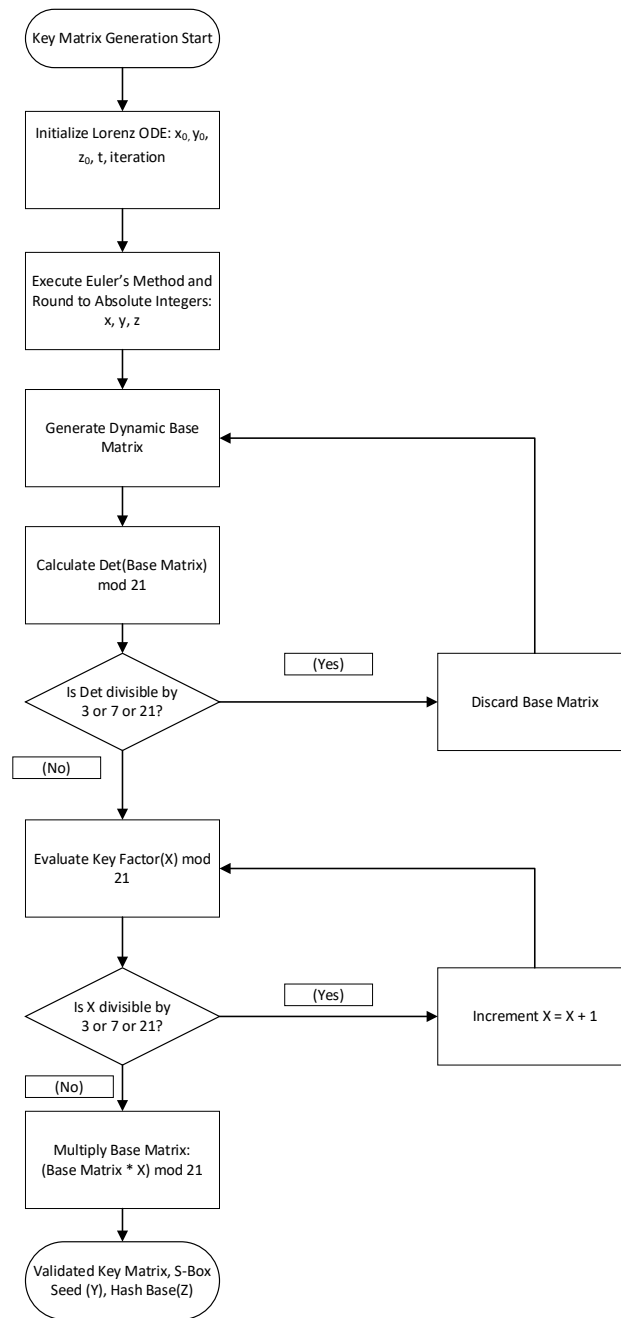


Figure 8 Flowchart of Key Matrix Generation and Two-Step Validation

ALGORITHM 8: CHAOTIC KEY FACTOR AND MATRIX GENERATION**Input:** BaseMatrix, x_0 , y_0 , z_0 , α , γ , β , Iterations, step(t)**Output:** Validated KeyMatrix, KeyFactor (X)

```
1:  // Stage 1: Lorenz ODE Chaos Generation
2:   $x \leftarrow x_0, y \leftarrow y_0, z \leftarrow z_0$ 
3:  FOR i = 1 to Iterations DO
4:       $L_x \leftarrow \alpha * (y - x)$ 
5:       $L_y \leftarrow (x * (\gamma - z)) - y$ 
6:       $L_z \leftarrow (x * y) - (\beta * z)$ 
7:       $x \leftarrow x + (L_x * t)$ 
8:       $y \leftarrow y + (L_y * t)$ 
9:       $z \leftarrow z + (L_z * t)$ 
10: END FOR
11:  $X \leftarrow \text{AbsoluteValue}(\text{Round}(X))$ 
12:  $Y \leftarrow \text{AbsoluteValue}(\text{Round}(Y))$ 
13:  $Z \leftarrow \text{AbsoluteValue}(\text{Round}(Z))$ 
14: // Stage 2: Base Matrix Validation
15: IsValidBase  $\leftarrow$  False
16: WHILE IsValidBase == False DO
17:     BaseMatrix  $\leftarrow$  GenerateRandomBaseMatrix()
18:     Det(B)  $\leftarrow$  CalculateDeterminant(BaseMatrix) mod 21
19:     IF (Det mod 3  $\neq$  0) AND (Det mod 7  $\neq$  0) AND (Det mod 21  $\neq$  0) THEN
20:         IsValidBase  $\leftarrow$  True
21:     END IF
22: END WHILE
23: // Stage 3: Key Factor Validation
24: IsValidKeyFactor  $\leftarrow$  False
25: WHILE IsValidKeyFactor == False DO
26:     IF (X mod 3  $\neq$  0) AND (X mod 7  $\neq$  0) AND (X mod 21  $\neq$  0) THEN
```

```
27:             IsValidKeyFactor  $\leftarrow$  True
28:         ELSE
29:              $X \leftarrow X + 1$ 
30:         END IF
31:     END WHILE
32: // Stage 4: Matrix Scaling and Bounding
33: ValidatedKeyMatrix  $\leftarrow$  MultiplyScalar(BaseMatrix, X)
34: ValidatedKeyMatrix  $\leftarrow$  ApplyModulo(ValidatedKeyMatrix, 21)
35: RETURN ValidatedKeyMatrix, KeyFactor (X)
```

e) Algorithm 9: Chaotic Seed Extraction and Assignment

Algorithm 9 rounds the final Lorenz floating-point coordinates into integers to serve as the S-Box seed and the rolling hash base. It integrates the continuous-time chaos into discrete integer values that the system can use for block-level obfuscation.

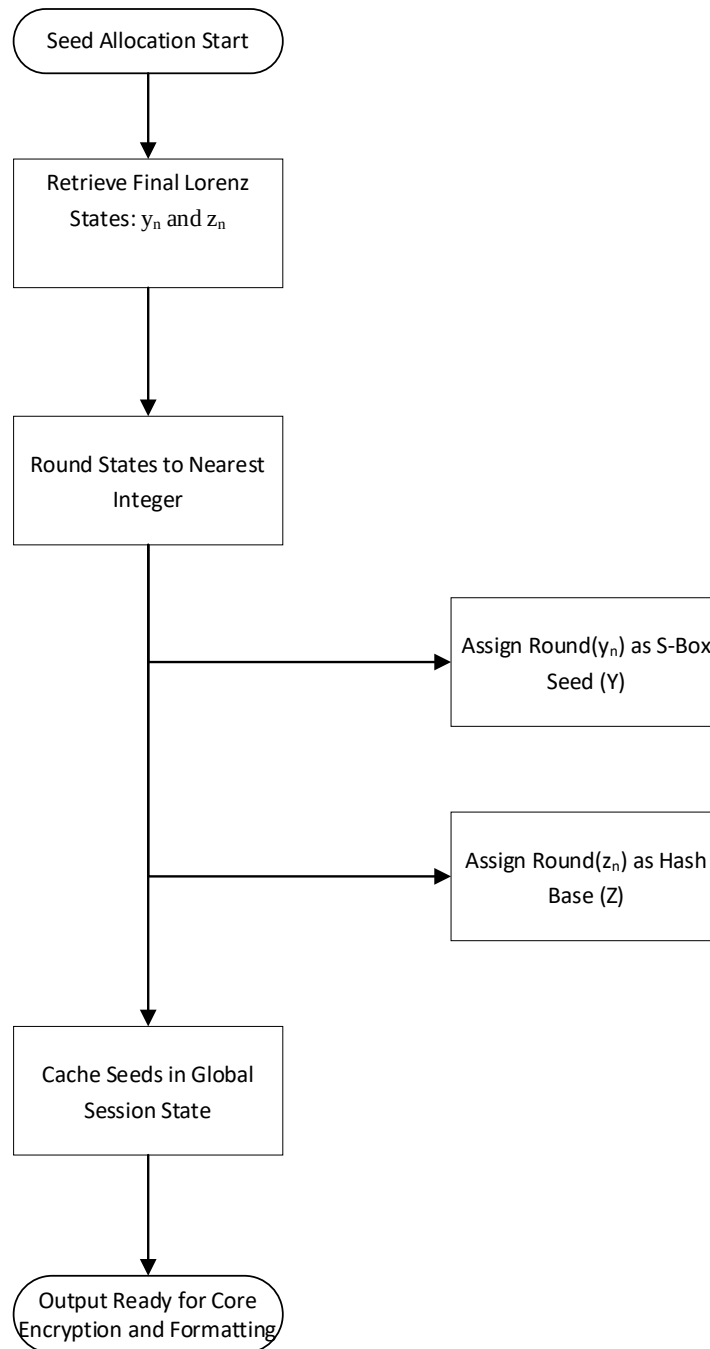


Figure 9 Flowchart of Entropy Seed Allocation

ALGORITHM 9: CHAOTIC SEED EXTRACTION AND ASSIGNMENT**Input:** Final Lorenz ODE Coordinates (y_{final} , z_{final})**Output:** S-Box Seed (Y), Rolling Hash Base (Z)

- 1: // Stage 1: Rounding Extraction
- 2: $\text{Integer}(Y) \leftarrow \text{RoundToInt}(y_{\text{final}})$
- 3: $\text{Integer}(Z) \leftarrow \text{RoundToInt}(z_{\text{final}})$
- 4: // Stage 2: Session Assignment
- 5: $\text{GlobalSession.SBoxSeed} \leftarrow \text{Integer}(Y)$
- 6: $\text{GlobalSession.HashBase} \leftarrow \text{Integer}(Z)$
- 7: RETURN $\text{Integer}(Y)$, $\text{Integer}(Z)$

f) Algorithm 10: Chaotic S-Box Execution

Algorithm 10 scrambles the matrix values non-linearly by using an array shuffled by a S-Box diffusion. It destructs any remaining linear relationships within the plaintext, and it heavily reinforcing the system's algebraic security.

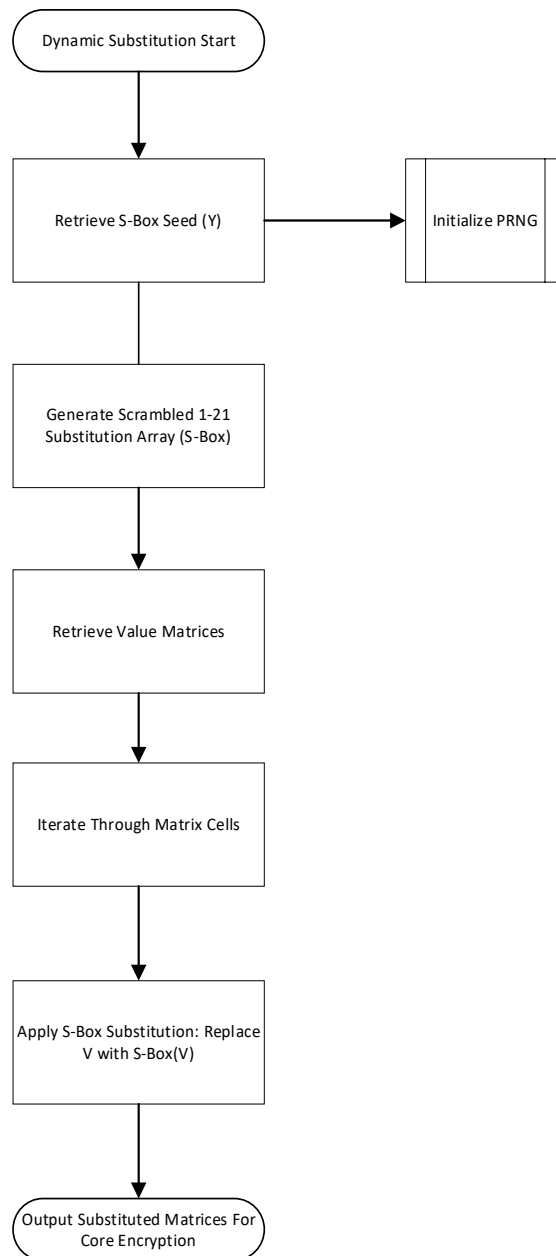


Figure 10 Flowchart of Dynamic Substitution

ALGORITHM 10: CHAOTIC S-BOX EXECUTION**Input:** ValueList (Plaintext Matrices), SBoxSeed(Y)**Output:** SubstitutedValueList

```
1: // Initialize Chaotic S-Box
2: Initialize SBox ← Array [1 to 21]
3: PRNG ← InitializeRandomGenerator(Seed: SBoxSeed(Y))
4: SBox ← ScrambleArray(SBox, PRNG)
5: Initialize SubstitutedValueList ← Empty List
6: // Non-Linear Substitution Loop
7: FOR EACH Matrix IN ValueList DO
8:     Create SubstitutedMatrix [Rows, Cols]
9:     FOR r = 0 to Rows – 1 DO
10:        FOR c = 0 to Cols – 1 DO
11:            OriginalValue ← Matrix[r, c]
12:            IF OriginalValue > 0 THEN
13:                SubstitutedMatrix[r, c] ← SBox[OriginalValue]
14:            ELSE
15:                SubstitutedMatrix[r, c] ← 0 // Preserve Matrix Padding
16:            END IF
17:        END FOR
18:    END FOR
19:    SubstitutedValueList.Append(SubstitutedMatrix)
20: END FOR
21: RETURN SubstitutedValueList
```

g) Algorithm 11: Hill Cipher Matrix Multiplication

Algorithm 11 multiplies the substituted plaintext matrices by the chaotic key matrix under strict modulo-21 boundaries. This is the core diffusion process which successfully scrambles the massive data payloads in linear time without symmetric delegation.

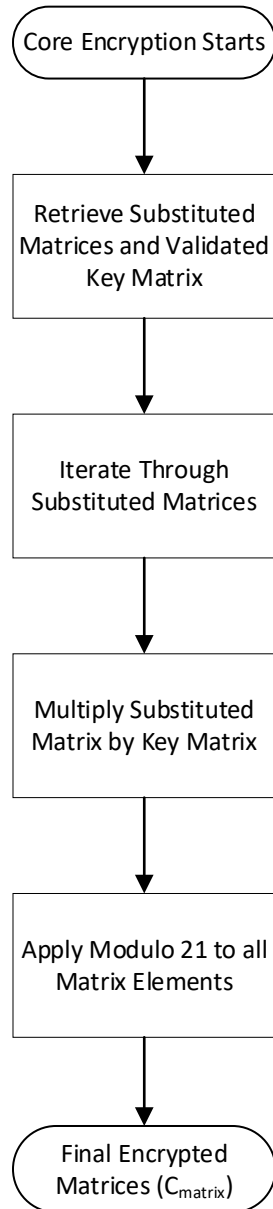


Figure 11 Flowchart of Hill Cipher Multiplication

ALGORITHM 11: HILL CIPHER MATRIX MULTIPLICATION**Input:** SubstitutedValueList, KeyMatrix**Output:** SubstitutedValueList (Ciphertext Matrices)

```
1: Initialize EncryptedValueList ← Empty List
2: FOR EACH SubstitutedMatrix IN SubstitutedValueList DO
3:     // Linear Hill Cipher Multiplication
4:     EncryptedMatrix ← MultiplyMatrices(KeyMatrix, SubstitutedMatrix)
5:     Rows ← GetRowCount(EncryptedMatrix)
6:     Cols ← GetColCount(EncryptedMatrix)
7:     // Apply Modulo 21 Bounds
8:     FOR r = 0 to Rows – 1 DO
9:         FOR c = 0 to Cols – 1 DO
10:            EncryptedMatrix[r, c] ← EncryptedMatrix[r, c] mod 21
11:        END FOR
12:    END FOR
13:    EncryptedValueList.Append(EncryptedMatrix)
14: END FOR
15: RETURN EncryptedValueList
```

h) Algorithm 12: Rolling Hash Tagging and Shuffle

Algorithm 12 computes a sequential rolling hash for each matrix and completely diffuses the order of the deck using a Fisher-Yates shuffle. It provides immense structural obfuscation, ensuring attackers cannot map geometric patterns from contiguous repeating characters.

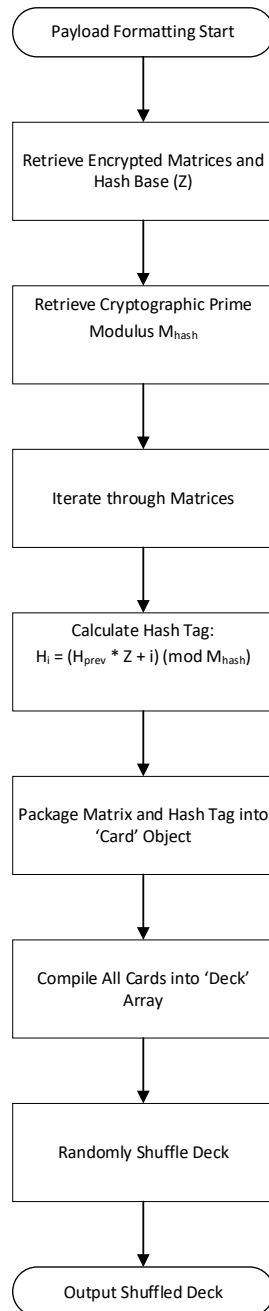


Figure 12 Flowchart of Deck Tagging and Shuffling

ALGORITHM 12: ROLLING HASH TAGGING AND SHUFFLE**Input:** EncryptedValueList, HashBase(Z), HashModulus**Output:** ShuffledDeck (Array of Cards)

```
1: Initialize Deck  $\leftarrow$  Empty Array
2: PreviousHash  $\leftarrow$  0
3: FOR i = 1 to length(EncryptedValueList) DO
4:     CurrentMatrix  $\leftarrow$  EncryptedValueList[i - 1]
5:     // Calculate Large-Scale Rolling Hash
6:     CurrentHash  $\leftarrow$  (PreviousHash * Hashbase(Z) + i) mod HashModulus
7:     // Encapsulate and Store
8:     NewCard  $\leftarrow$  CreateCard(Data: CurrentMatrix, Tag: CurrentHash)
9:     Deck.Append(NewCard)
10:    PreviousHash  $\leftarrow$  CurrentHash
11: END FOR
12: ShuffledDeck  $\leftarrow$  FisherYatesShuffle(Deck)
13: RETURN ShuffledDeck
```

i) Algorithm 13: Subset-Sum Signature Generation

Algorithm 13 produces a binary signature vector by greedily resolving the encapsulated header against the sender's private key. It binds the payload to the sender's identity using NP-hard mathematics, ensuring quantum-resistant non-repudiation.

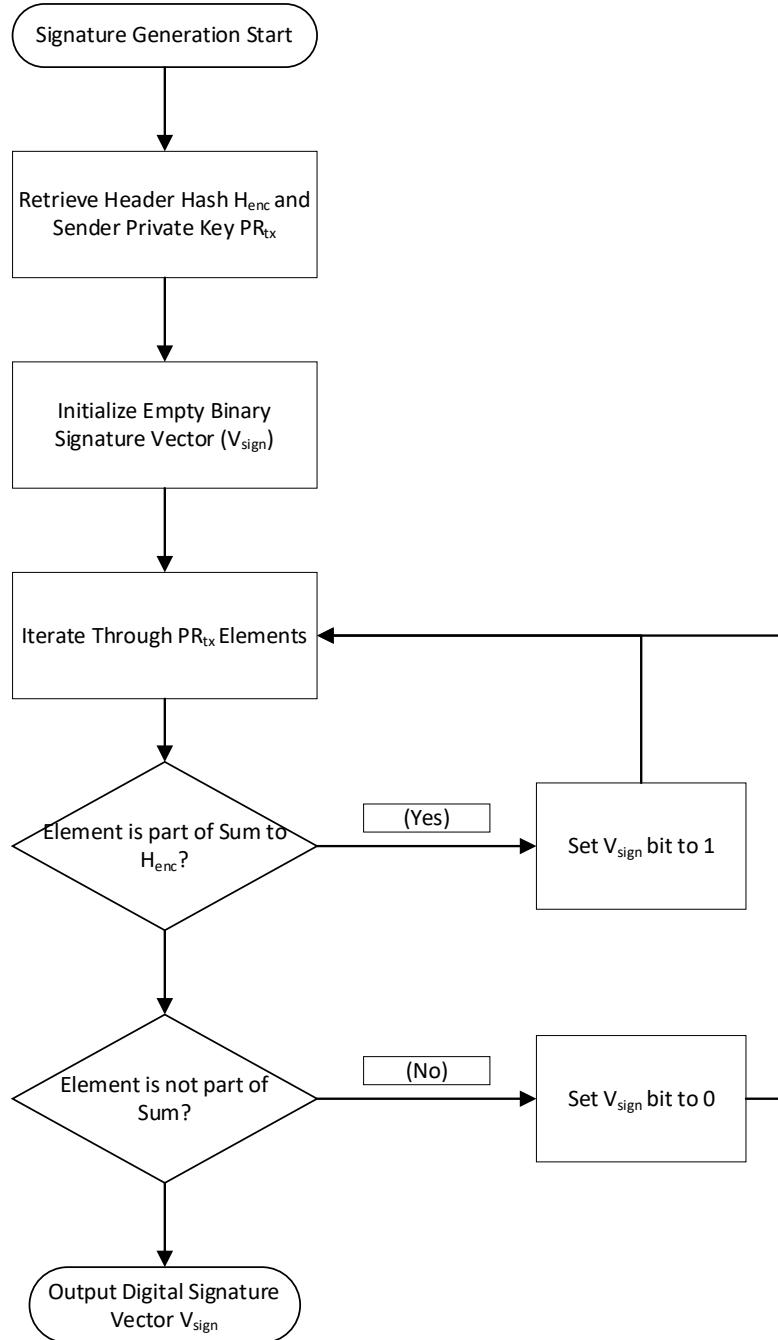


Figure 13 Flowchart of Digital Signature Generation

ALGORITHM 13: SUBSET-SUM SIGNATURE GENERATION**Input:** SenderPrivateKey(PR_{tx}), TargetHeader(H_{enc})**Output:** SignatureVector(V_{sig})

```
1: Initialize  $V_{sig} \leftarrow$  Empty Binary Array of size  $\text{length}(PR_{tx})$ 
2: TargetRemainder  $\leftarrow$  TargetHeader
3: // Subset-Sum Evaluation (Knapsack resolution)
4: FOR  $i = \text{length}(PR_{tx}) - 1$  DOWNT0 0 DO
5:     IF  $PR_{tx}[i] \leq \text{TargetRemainder}$  THEN
6:          $V_{sig}[i] \leftarrow 1$ 
7:         TargetRemainder  $\leftarrow$  TargetRemainder -  $PR_{tx}[i]$ 
8:     ELSE
9:          $V_{sig}[i] \leftarrow 0$ 
10:    END IF
11: END FOR
12: // Verification Check
13: IF TargetRemainder  $\neq 0$  THEN
14:     HALT Execution (Error: "Invalid Trapdoor Resolution")
15: END IF
16: RETURN  $V_{sig}$ 
```

j) Algorithm 14: Payload Serialization

Algorithm 14 appends the header, signature, and shuffled matrices into a single data structure and serializes it for network transmission. It packages the theoretical cryptography into a deployable, real-world format for cross-platform network routing.

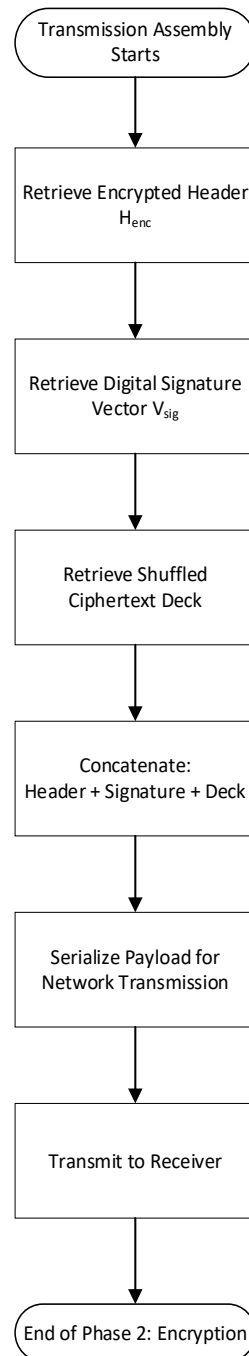


Figure 14 Flowchart of Payload Assembly and Transmission

ALGORITHM 14: PAYLOAD SERIALIZATION**Input:** Header (H_{enc}), Signature (V_{sig}), Deck (ShuffledDeck)**Output:** TransmittablePayload

- 1: Initialize TransmittablePayload $\leftarrow$ New Data Structure
- 2: // Append sequential cryptographic blocks
- 3: TransmittablePayload.Append(H_{enc})
- 4: TransmittablePayload.Append(V_{sig})
- 5: TransmittablePayload.Append(ShuffledDeck)
- 6: // Serialize for network transmission (e.g., JSON or Byte Array in XML tree format)
- 7: SerializedData $\leftarrow$ Serialize(TransmittablePayload)
- 8: TransmitToNetwork(SerializedData)
- 9: RETURN Status_Success

3) Phase 3: Validation and Integrity

a) Algorithm 15: Digital Signature Verification

Algorithm 15 computes a checksum via dot product and uses a modular inverse transformation against the sender's public key to verify integrity. It serves as an immediate quantum-resistant fail-safe, dropping tampered or spoofed payloads before they consume the receiver's CPU power.

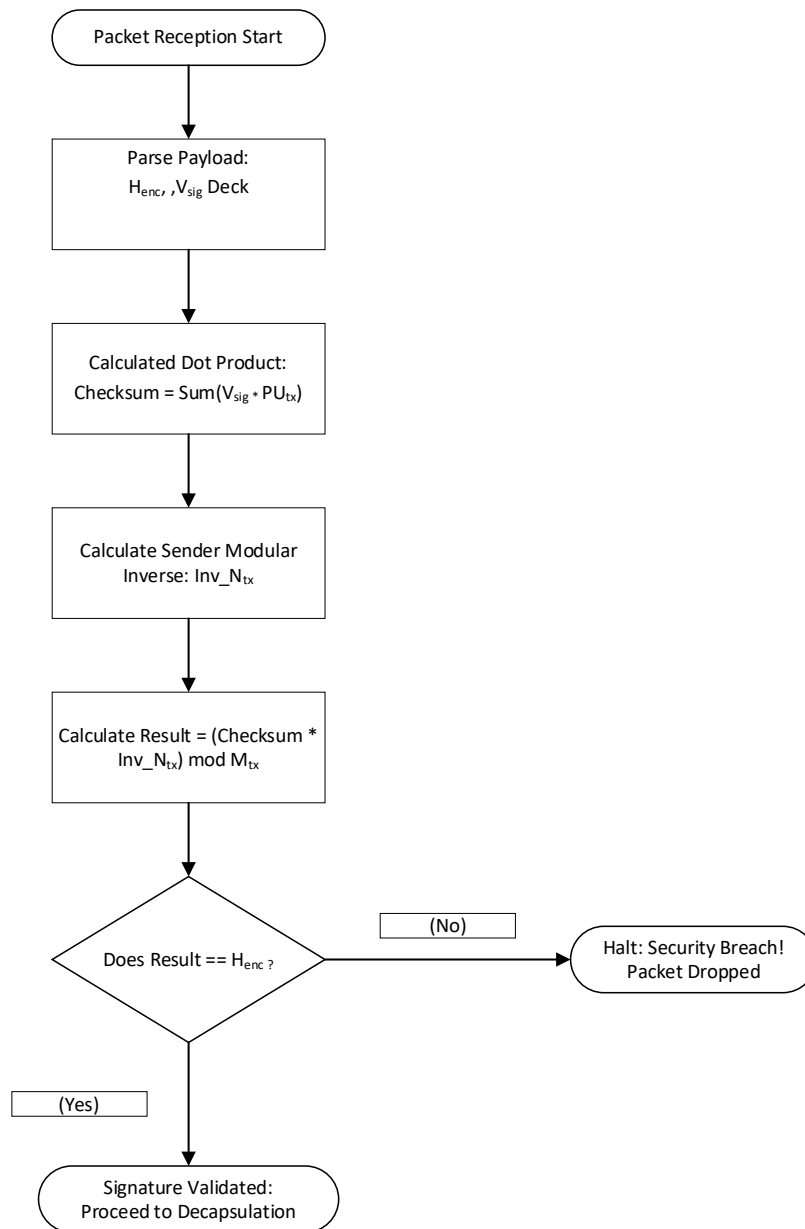


Figure 15 Flowchart of Signature Verification

ALGORITHM 15: DIGITAL SIGNATURE VERIFICATION

Input: ReceivedHeader (H_{enc}), ReceivedSignature (V_{sig}), SenderPublicKey (PU_{tx}), SenderMod (M_{tx}), SenderMultiplier (N_{tx})

Output: Boolean (IsValid)

```
1: Initialize Checksum  $\leftarrow 0$ 
2: // Calculate Public Checksum via Dot Product
3: FOR  $i = 0$  to  $\text{length}(V_{sig}) - 1$  DO
4:     Checksum  $\leftarrow$  Checksum + ( $V_{sig}[i] * PU_{tx}[i]$ )
5: END FOR
6: // Inverse Modulo Transformation
7:  $Inv_N \leftarrow \text{CalculateModularInverse}(N_{tx}, M_{tx})$ 
8: VerificationResult  $\leftarrow$  (Checksum *  $Inv_N$ ) mod  $M_{tx}$ 
9: // Integrity Check
10: IF VerificationResult ==  $H_{enc}$  THEN
11:     RETURN True
12: ELSE
13:     RETURN False
14: END IF
```

b) Algorithm 16: Subset-Sum Key Decapsulation

Algorithm 16 uses a modular inverse and greedy subset-sum resolution to break the trapdoor and extract the original binary session vector. It successfully rebuilds the chaotic starting parameters on the receiver's end without relying on traditional key exchange mechanisms.

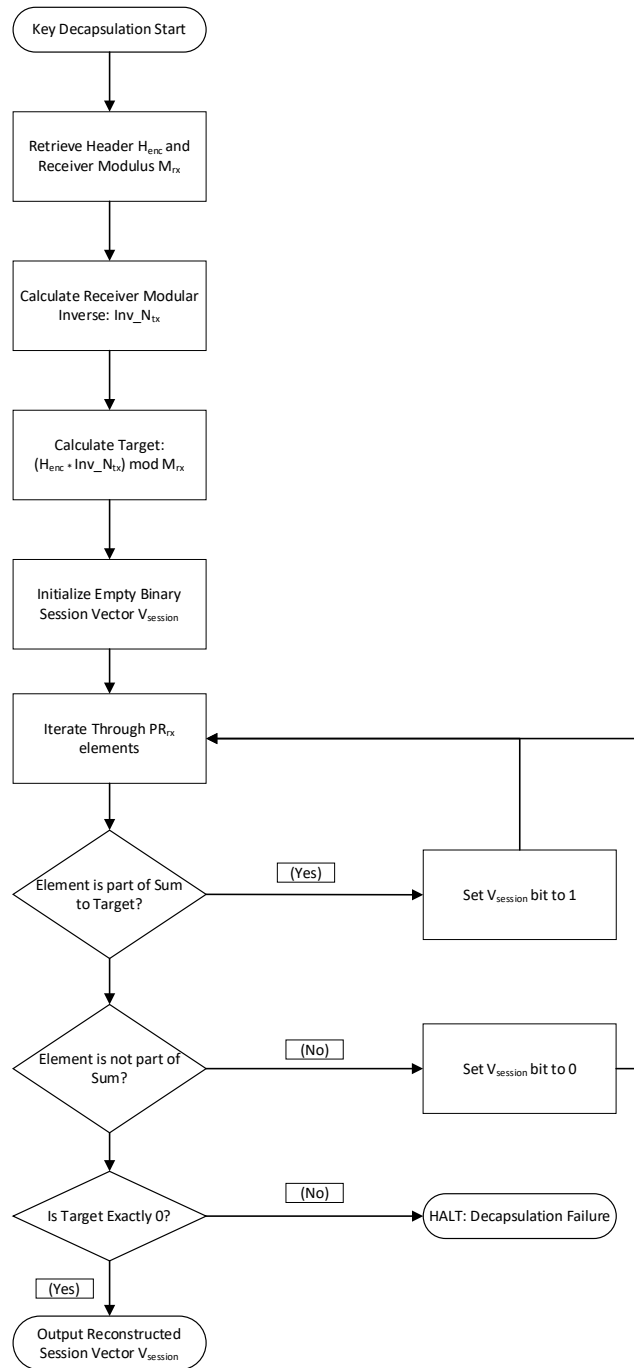


Figure 16 Flowchart of Key Decapsulation

ALGORITHM 16: SUBSET-SUM KEY DECAPSULATION**Input:** ReceivedHeader (H_{enc}), ReceiverPrivateKey (PR_{rx}), ReceiverMod (M_{rx}), ReceiverMultiplier (N_{rx})**Output:** SessionVector (V_{session})

```
1:   $Inv_N \leftarrow \text{CalculateModularInverse}(N_{\text{rx}}, M_{\text{rx}})$ 
2:   $\text{TargetSum} \leftarrow (H_{\text{enc}} * Inv_N) \bmod M_{\text{rx}}$ 
3:  Initialize  $V_{\text{session}} \leftarrow$  Empty Binary Array of Size length ( $PR_{\text{rx}}$ )
4:   $\text{Checksum} \leftarrow \text{TargetSum}$ 
5:  // Solve Trapdoor via Greedy Resolution
6:  FOR  $i = \text{length}(PR_{\text{rx}}) - 1$  DOWNT0 0 DO
7:      IF  $PR_{\text{rx}}[i] \leq \text{CurrentSum}$  THEN
8:           $V_{\text{session}}[i] \leftarrow 1$ 
9:           $\text{CurrentSum} \leftarrow \text{CurrentSum} - PR_{\text{rx}}[i]$ 
10:     ELSE
11:          $V_{\text{session}}[i] \leftarrow 0$ 
12:     END IF
13: END FOR
14: // Integrity Verification
15: IF  $\text{CurrentSum} \neq 0$  THEN
16:     HALT Execution (Error: "Trapdoor Decapsulation Failed")
17: END IF
18: RETURN  $V_{\text{session}}$ 
```

c) Algorithm 17: Payload Sorting and Key Regeneration

Algorithm 17 uses Introsort to reorder the randomly shuffled deck based on expected hashes, while simultaneously regenerating the validated key matrix. It resolves the system's major limitation of ciphertext expansion by hyper-accelerating sequence reconstruction to achieve high "True Throughput".

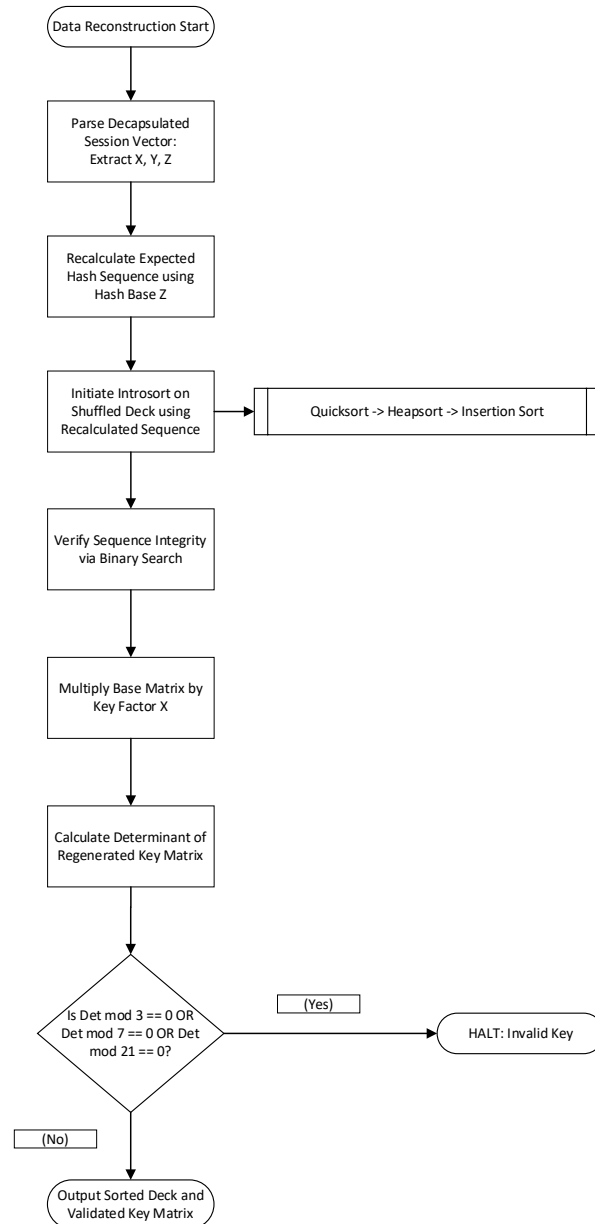


Figure 3-17 Flowchart of Payload Sorting and Key Generation

ALGORITHM 17: PAYLOAD SORTING AND KEY REGENERATION**Input:** ShuffledDeck, HashModulus, KeyFactor(X), BaseMatrix(Y), HashBase(Z)**Output:** SortedDeck, ValidatedKeyMatrix

```
1: // Stage 1: Expected Sequence Generation
2: ExpectedHashes ← Empty Array
3: PreviousHash ← 0
4: FOR i = 1 to length(ShuffledDeck) DO
5:     CurrentHash ← (PreviousHash * HashBase(Z) + i) mod HashModulus
6:     ExpectedHashes.Append(CurrentHash)
7:     PreviousHash ← CurrentHash
8: END FOR
9: // Stage 2: Introsort Execution
10: RecursionLimit ← 2 * Log2(length(ShuffledDeck))
11: SortedDeck ← Introsort(ShuffledDeck, ExpectedHashes, RecursionLimit)
12: // Stage 3: Sequence Integrity Check
13: FOR EACH Hash IN ExpectedHashes DO
14:     IF BinarySearch(SortedDeck, Hash) == False THEN
15:         HALT Execution (Error: "Missing Card in Sequence")
16:     END IF
17: END FOR
18: // Stage 4: Key Matrix Regeneration
19: TempMatrix ← MultiplyScalar(BaseMatrix, KeyFactor(X))
20: Det ← CalculateDeterminant(TempMatrix)
21: IF (Det mod 3 == 0) OR (Det mod 7 == 0) OR (Det mod 21 == 0) THEN
22:     HALT Execution (Error: "Invalid Regenerated Matrix")
23: END IF
24: ValidatedKeyMatrix ← TempMatrix
25: RETURN SortedDeck, ValidatedKeyMatrix
```

4) Phase 4: Decryption

a) Algorithm 18: Decryption and Reverse Substitution

Algorithm 18 calculates the adjugate and modular inverse of the key matrix, reverses the Hill cipher multiplication, and resolves the zero-state edge cases before doing a reverse S-Box lookup. It proves that the localized GF(21) matrix diffusion is perfectly reversible and mathematically sound, despite the chaotic mutations.

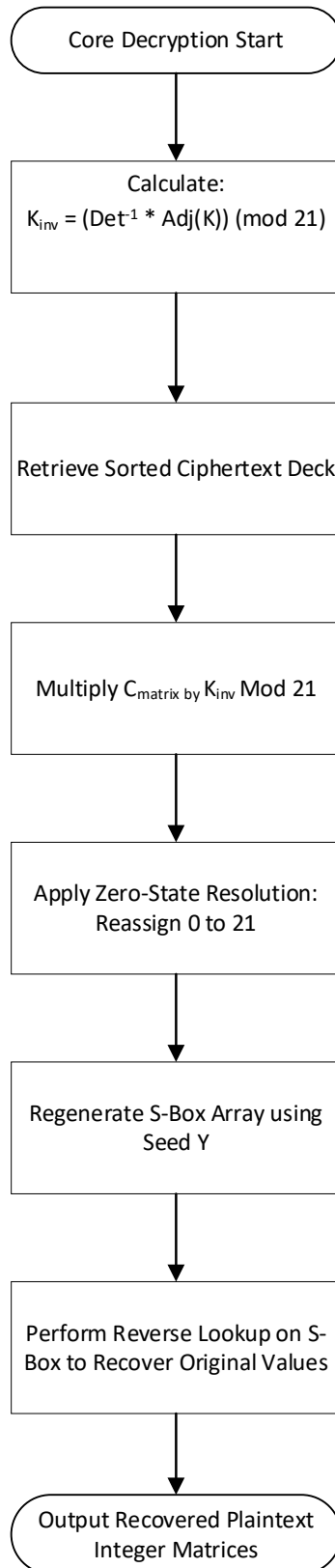


Figure 18 Flowchart of Core Decryption Process

ALGORITHM 18: DECRYPTION AND REVERSE SUBSTITUTION**Input:** SortedDeck, ValidatedKeyMatrix, SBoxSeed(Y)**Output:** RecoveredValueList

```
1: // Stage 1: Generate Inverse Key Matrix
2: Det ← CalculateDeterminant(ValidatedKeyMatrix)
3: DetInv ← ModularInverse(Det, 21)
4: AdjugateK ← CalculateAdjugate(ValidatedKeyMatrix)
5: KeyMatrixInverse ← (DetInv * AdjugateK) mod 21
6: // Stage 2: Regenerate Chaotic S-Box
7: Initialize SBox ← Array [1 to 21]
8: PRNG ← InitializeRandomGenerator(Seed: SBoxSeed(Y))
9: SBox ← ScrambleArray(SBox, PRNG)
10: Initialize RecoveredValueList ← Empty List
11: // Stage 3: Decryption Loop
12: FOR EACH Card IN SortedDeck DO
13:     Cmatrix ← Card.Data
14:     // Inverse Hill Cipher Multiplication
15:     Ematrix ← MultiplyMatrices(Cmatrix, KeyMatrixInverse)
16:     Rows ← GetRowCount(Ematrix)
17:     Cols ← GetColCount(Ematrix)
18:     Create Vmatrix [Rows, Cols]
19:     // Zero-State Resolution and Reverse Substitution
20:     FOR r = 0 to Rows – 1 DO
21:         FOR c = 0 to Cols – 1 DO
22:             Ematrix[r, c] ← Ematrix[r, c] mod 21
23:             // Zero-State Edge Case (0 equals 21)
24:             IF Ematrix[r, c] == 0 THEN
25:                 Ematrix[r, c] ← 21
26:             END IF
27:             // Reverse S-Box Lookup
```



```
28:             IF  $E_{\text{matrix}}[r, c] > 0$  THEN
29:                  $V_{\text{matrix}}[r, c] \leftarrow \text{FindIndexOf}(\text{SBox}, E_{\text{matrix}}[r, c])$ 
30:             ELSE
31:                  $V_{\text{matrix}}[r, c] \leftarrow 0$  // Ignore Padding
32:             END IF
33:         END FOR
34:     END FOR
35:     RecoveredValueList.Append( $V_{\text{matrix}}$ )
36: END FOR
37: RETURN RecoveredValueList
```

b) Algorithm 19: Demapping and Plaintext Assembly

Algorithm 19 reverses both Caesar shifts, reattaches the isolated formatting markers from phase 1, and converts the mapped numbers back into the original English text. It ensures the absolute semantic integrity, proving that the Enochian Cryptosystem can protect and perfectly restore highly complex enterprise data structures.

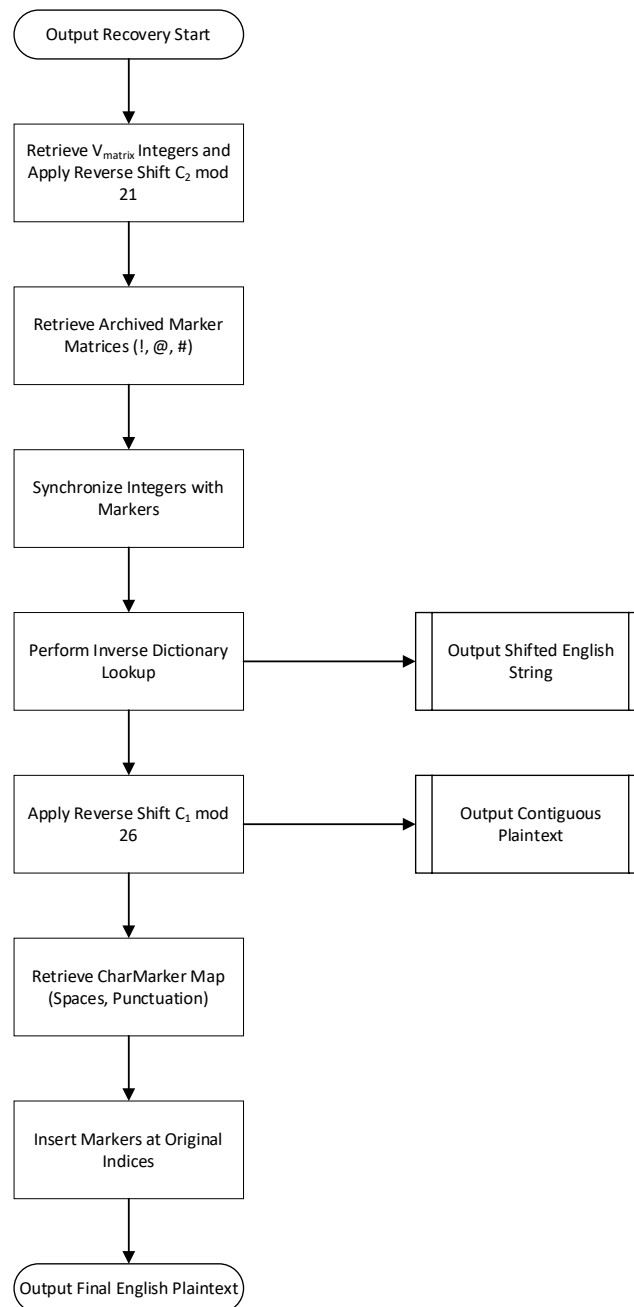


Figure 19 Flowchart of Output Recovery

ALGORITHM 19: DEMAPPING AND PLAINTEXT ASSEMBLY**Input:** RecoveredValueList, MarkerList, C_1 , C_2 , CharMarkerMap, InverseEnochianDictionary**Output:** FinalPlaintextString

```
1: Initialize ShiftedEnochianArray  $\leftarrow$  Empty Array
2: // Stage 1: Flatten Matrices and Reverse  $C_2$  Shift
3: FOR EACH Matrix IN RecoveredValueList DO
4:     FOR EACH Cell (S) IN Matrix DO
5:         IF Cell > 0 THEN
6:              $V \leftarrow ((S - 1 - C_1 + 21) \bmod 21) + 1$ 
7:             ShiftedEnochianArray.Append(V)
8:         END IF
9:     END FOR
10: END FOR
11: // Stage 2: Synchronize Markers and Demapping
12: Initialize ShiftedEnglishString  $\leftarrow$  ""
13: FOR i = 0 to length(ShiftedEnochianArray) - 1 DO
14:     NumValue  $\leftarrow$  ShiftedEnochianArray[i]
15:     Marker  $\leftarrow$  ExtractCorrespondingMarker(MarkerList, i)
16:     QueryKey  $\leftarrow$  NumValue + Marker
17:     EnglishChar  $\leftarrow$  Lookup(InverseEnochianDict, QueryKey)
18:     ShiftedEnglishString  $\leftarrow$  ShiftedEnglishString + EnglishChar
19: END FOR
20: // Stage 3: Reverse  $C_1$  Shift
21: Initialize CleanPlaintext  $\leftarrow$  ""
22: FOR i = 0 to length(ShiftedEnglishString) - 1 DO
23:     CharIndex  $\leftarrow$  ShiftedEnglishString[i] - 'A'
24:     OriginalIndex  $\leftarrow$  (CharIndex -  $C_1$  + 26) mod 26
25:     CleanPlaintext  $\leftarrow$  CleanPlaintext + (OriginalIndex + 'A')
```

```
26: END FOR
27: // Stage 4: Structural Reassembly
28: FinalPlaintextString ← InjectMarkers(CleanPlaintext, CharMarkerMap)
29: FinalPlaintextString ← ConvertWordsToNumbers(FinalPlaintextString)
30: RETURN FinalPlaintextString
```